LOCAL LLM HOSTING AND EVALUATION

Local AI Lab: Two Text Generation Models, One Streamlit Interface, No Cloud AI API

Project title	Local LLM Hosting and Evaluation (Local AI Lab)
Project ID	EF101 P16
Course	ITFA (EF-101)
Group members	SAMAN, DUA TOSEEF, SHAFIA EEMAN
Student IDs	EE-52, EE-53, EE-59
Instructor	DR. FEZAN RAFIQ
Section	B Group number: 16
Submission date	15-11-2026
GitHub repository	https://github.com/saman294/EF101-Local-LLM-Project

2. Objective

This project hosts two small text generation models locally on a Windows laptop using Ollama, connects them to a custom web interface built with Python and Streamlit, and evaluates how the software and hardware behave while the models run. No cloud AI API is used at any point: the prompts, the models and the answers all stay on the local machine.

The project is treated as a complete computing system rather than only an AI demo. The group therefore observes the operating system, the Python environment, the model runtime, storage use, memory use, processor use and response time together, and compares two models of similar size but different design.

Specific goals:

- Install and verify Python, pip and Ollama on Windows.
- Download two local models (llama3.2:1b and qwen2.5:1.5b) and inspect their parameter counts, sizes and quantization.
- Build a Streamlit interface with a model switcher, conversation clearing, response timing and a saved conversation log.
- Measure response time, model load time, RAM, CPU and GPU behaviour using a fixed set of prompts.
- Demonstrate offline operation and test how the system handles failures.

3. System Specification

The project was developed and tested on a single Windows laptop. The details below are taken from Windows Settings, About (Figures 1 and 2) and the Command Prompt banner (Figure 3).

Item	Value
Operating system	Windows 10 Pro, 64-bit, x64-based processor
Windows version	22H2 (installed 3/18/2023)
OS build	19045.6466 (Command Prompt reports version 10.0.19045.6466)
Storage	119 GB SSD, SAMSUNG MZ7LN128HCHP-000L1
Processor (CPU)	Intel(R) Core(TM) i5-6300U CPU @ 2.40GHz 2.50GHz
Installed RAM	8.00 GB
GPU	Intel(R) HD Graphics 520 (128 MB)

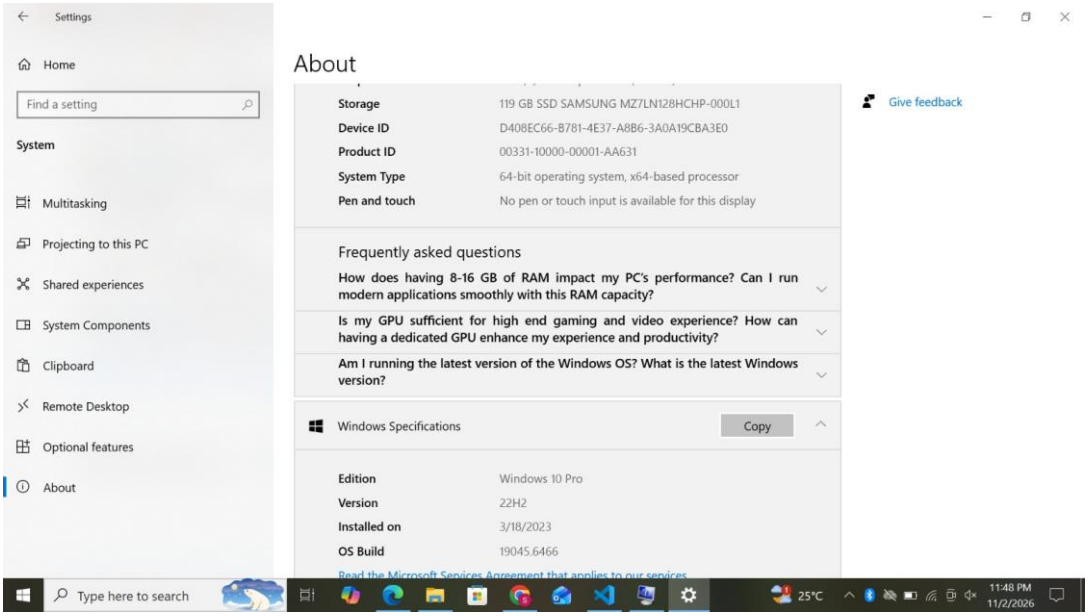


Figure 1: Windows Settings > About: device storage and Windows specifications (Windows 10 Pro 22H2, build 19045.6466).

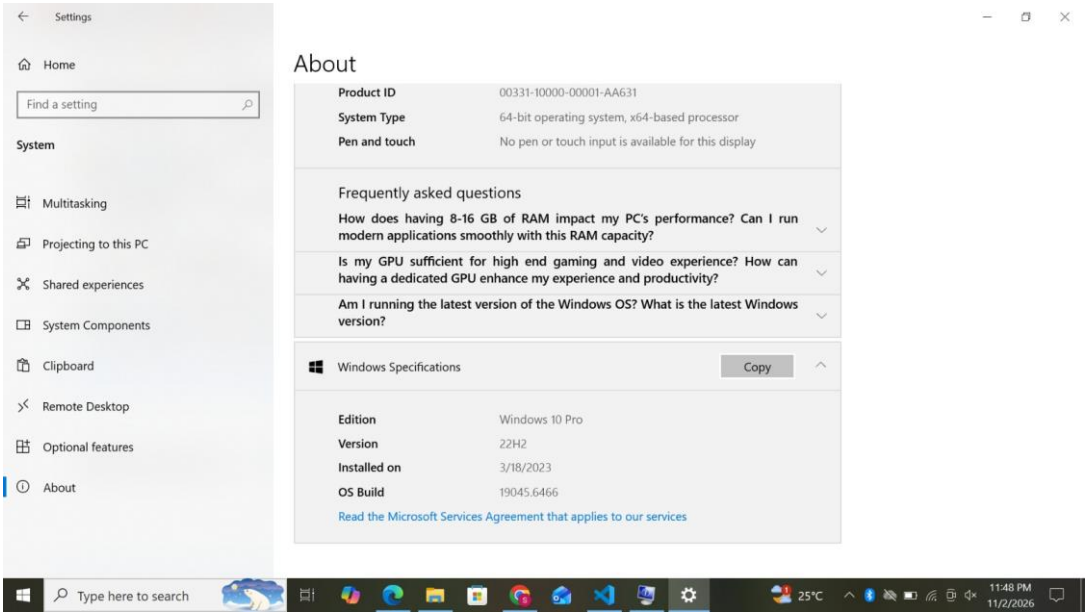


Figure 2: Windows Settings > About: system type and Windows specifications.

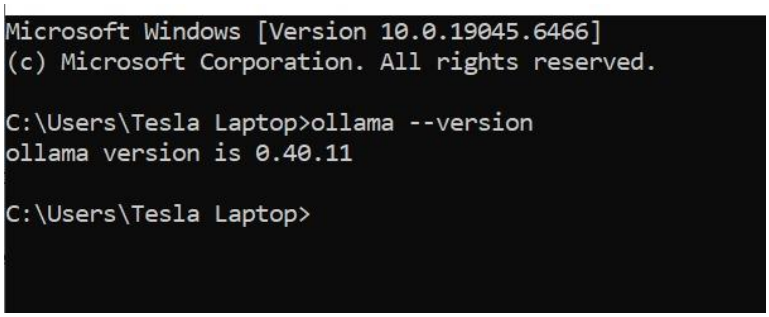


Figure 3: Command Prompt showing the Windows version and the installed Ollama version (0.40.11).

4. Tools and Model Details

4.1 Software tools

Tool	Version	Source
Python	3.12.10 (64-bit)	python.org, python-3.12.10-amd64.exe
Pip	25.0.1	Bundled with Python 3.12
Ollama	0.40.11	OllamaSetup.exe from the official Ollama GitHub release assets
Streamlit	1.66.0	Installed with pip from requirements.txt
requests	2.34.2	Installed with pip from requirements.txt
Code editor	Visual Studio Code	Microsoft

Python 3.12.10 was downloaded from https://www.python.org/ftp/python/3.12.10/python-3.12.10-amd64.exe. Ollama was installed with OllamaSetup.exe, downloaded from the official Ollama release assets on GitHub. Figure 4 confirms the Python and pip versions and Figure 3 confirms the Ollama version.

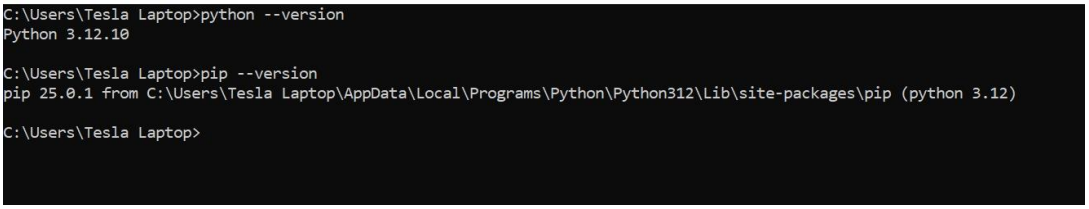


Figure 4: python --version and pip --version in Command Prompt (Python 3.12.10, pip 25.0.1).

4.2 Python dependencies

The project folder LocalLLMProject contains a requirements.txt file. Its first line is streamlit (the web interface) and its second line is requests (used to call the local Ollama service). Running python -m pip install -r requirements.txt reported every requirement as already satisfied, including the Streamlit dependencies such as altair, numpy, pandas, pillow, pydeck, protobuf, pyarrow, uvicorn, starlette, websockets and watchdog (Figures 5 to 8).

```
PS C:\Users\Tesla Laptop\Documents\LocalLLMProject> python -m pip inst
all -r requirements.txt
Requirement already satisfied: streamlit in c:\users\tesla laptop\appd
ata\local\programs\python\python312\lib\site-packages (from -r require
ments.txt (line 1)) (1.66.0)
Requirement already satisfied: requests in c:\users\tesla laptop\appda
ta\local\programs\python\python312\lib\site-packages (from -r requirem
ents.txt (line 2)) (2.34.2)
Requirement already satisfied: altair!=5.4.0,!=5.4.1,<7,>=5.0.0 in c:\
users\tesla laptop\appdata\local\programs\python\python312\lib\site-pa
ckages (from streamlit->-r requirements.txt (line 1)) (6.3.0)
Requirement already satisfied: click<9,>=7.0 in c:\users\tesla laptop\
appdata\local\programs\python\python312\lib\site-packages (from stream
lit->-r requirements.txt (line 1)) (8.5.0)
Requirement already satisfied: numpy<3,>=1.25.0 in c:\users\tesla lapt
op\appdata\local\programs\python\python312\lib\site-packages (from str
eamlit->-r requirements.txt (line 1)) (2.5.4)
Requirement already satisfied: packaging>=20 in c:\users\tesla laptop\
appdata\local\programs\python\python312\lib\site-packages (from stream
lit->-r requirements.txt (line 1)) (26.3)
Requirement already satisfied: pandas<4,>=1.5.3 in c:\users\tesla lapt
op\appdata\local\programs\python\python312\lib\site-packages (from str
eamlit->-r requirements.txt (line 1)) (3.0.6)
Requirement already satisfied: pillow<13,>=9.2.0 in c:\users\tesla lap
top\appdata\local\programs\python\python312\lib\site-packages (from st
reamlit->-r requirements.txt (line 1)) (12.3.0)
Requirement already satisfied: pydeck<1,>=0.8.0b4 in c:\users\tesla la
ptop\appdata\local\programs\python\python312\lib\site-packages (from s
treamlit->-r requirements.txt (line 1)) (0.9.3)
Requirement already satisfied: protobuf<8,>=5.26.1 in c:\users\tesla l
aptop\appdata\local\programs\python\python312\lib\site-packages (from
streamlit->-r requirements.txt (line 1)) (7.36.2)
```

Figure 5: Installing project requirements: python -m pip install -r requirements.txt (start of output).

```
Requirement already satisfied: pyarrow!=25.0.0,<27,>=10.0.1 in c:\user
s\tesla laptop\appdata\local\programs\python\python312\lib\site-packag
es (from streamlit->-r requirements.txt (line 1)) (26.0.0)
Requirement already satisfied: typing-extensions<5,>=4.10.0 in c:\user
s\tesla laptop\appdata\local\programs\python\python312\lib\site-packag
es (from streamlit->-r requirements.txt (line 1)) (4.16.0)
Requirement already satisfied: starlette<2,>=0.46.0 in c:\users\tesla
laptop\appdata\local\programs\python\python312\lib\site-packages (from
 streamlit->-r requirements.txt (line 1)) (1.8.0)
Requirement already satisfied: uvicorn<1,>=0.30.0 in c:\users\tesla la
ptop\appdata\local\programs\python\python312\lib\site-packages (from s
treamlit->-r requirements.txt (line 1)) (0.54.0)
Requirement already satisfied: anyio<5,>=4.0.0 in c:\users\tesla lapto
p\appdata\local\programs\python\python312\lib\site-packages (from stre
amlit->-r requirements.txt (line 1)) (4.15.1)
Requirement already satisfied: python-multipart<1,>=0.0.10 in c:\users
\tesla laptop\appdata\local\programs\python\python312\lib\site-package
s (from streamlit->-r requirements.txt (line 1)) (0.0.32)
Requirement already satisfied: websockets<18,>=12.0.0 in c:\users\tesl
a laptop\appdata\local\programs\python\python312\lib\site-packages (fr
om streamlit->-r requirements.txt (line 1)) (17.2)
Requirement already satisfied: itsdangerous<3,>=2.1.2 in c:\users\tesl
a laptop\appdata\local\programs\python\python312\lib\site-packages (fr
om streamlit->-r requirements.txt (line 1)) (2.2.0)
Requirement already satisfied: watchdog<7,>=2.1.5 in c:\users\tesla la
ptop\appdata\local\programs\python\python312\lib\site-packages (from s
treamlit->-r requirements.txt (line 1)) (6.0.0)
Requirement already satisfied: charset_normalizer<4,>=2 in c:\users\te
sla laptop\appdata\local\programs\python\python312\lib\site-packages (
from requests->-r requirements.txt (line 2)) (3.5.2)
Requirement already satisfied: idna<4,>=2.5 in c:\users\tesla laptop\a
```

Figure 6: Requirement check output (continued).

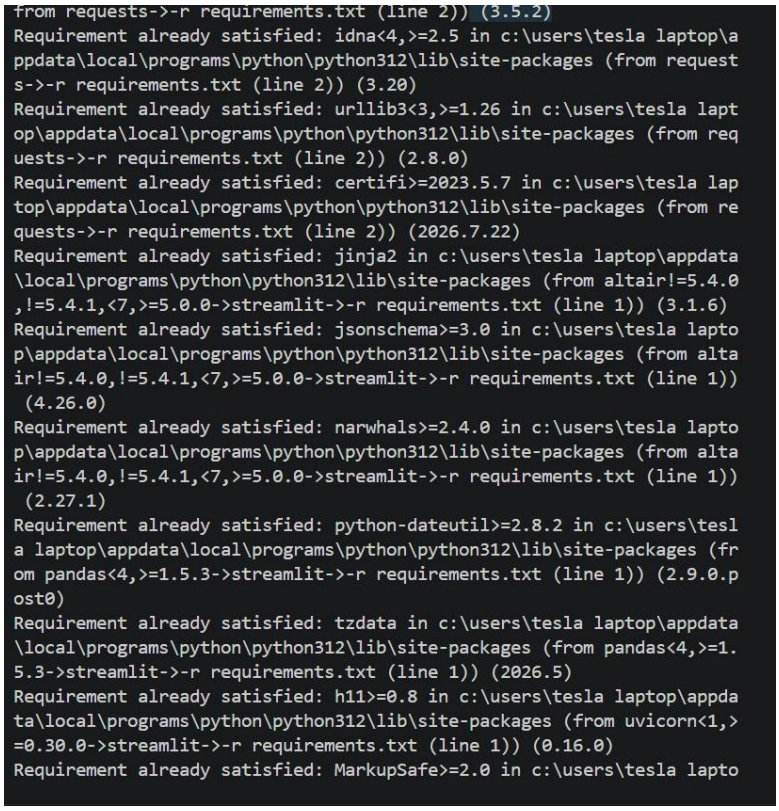


Figure 7: Requirement check output (continued).

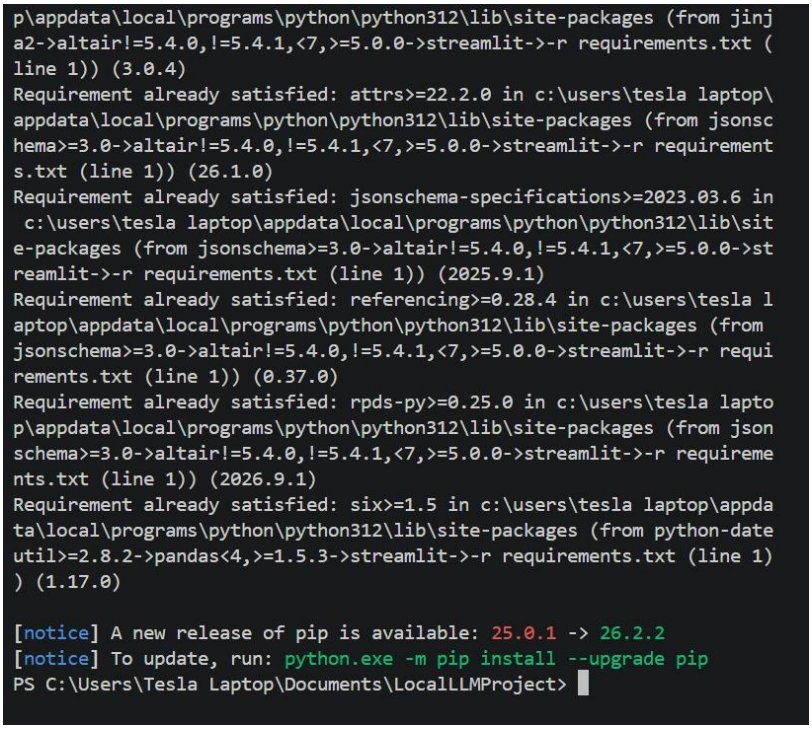


Figure 8: End of the output: all requirements satisfied; pip reports a newer release (25.0.1 to 26.2.2) is available.

4.3 Models

Two models were pulled from the Ollama model library with ollama pull. Figure 9 shows the qwen2.5:1.5b download: five layers were pulled, the sha256 digest was verified and the manifest was written successfully. Figure 10 shows the llama3.2:1b download in the same way, and Figure 11 lists both installed models.



Figure 9: ollama pull qwen2.5:1.5b completing successfully (986 MB main layer).

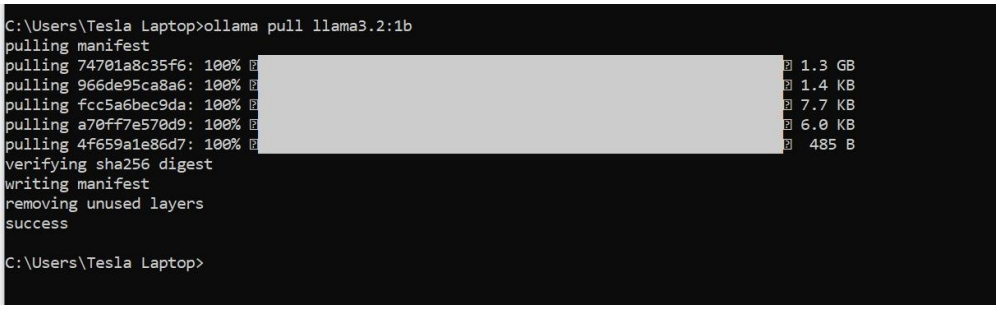


Figure 10: ollama pull llama3.2:1b completing successfully (1.3 GB main layer).

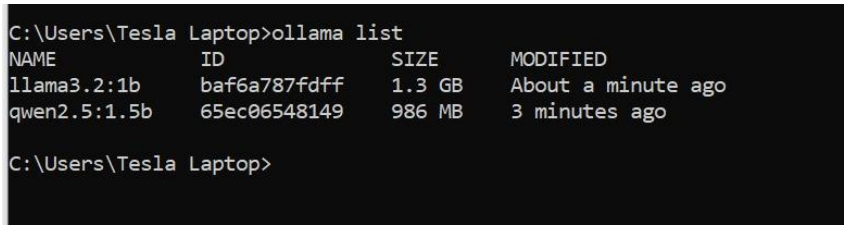


Figure 11: ollama list showing both installed models and their sizes.

The details below come from ollama show for each model (Figures 12 and 13).

Property	llama3.2:1b	qwen2.5:1.5b
Model ID	baf6a787fdff	65ec06548149
Developer / source	Meta, via the Ollama library	Alibaba Cloud (Qwen), via the Ollama library
Architecture	llama	qwen2
Parameters	1.2B	1.5B
Download size	1.3 GB	986 MB
Quantization	Q8_0	Q4_K_M
Context length	131072 tokens	32768 tokens
Embedding length	2048	1536
Capabilities	completion, tools	completion, tools
Licence	Llama 3.2 Community License Agreement (released September 25, 2024)	Apache License 2.0

```
C:\Users\Tesla Laptop>ollama show llama3.2:1b
  Model
    architecture        llama
    parameters          1.2B
    context length      131072
    embedding length    2048
    quantization        Q8_0
    template source     go_template

  Capabilities
    completion
    tools

  License
    LLAMA 3.2 COMMUNITY LICENSE AGREEMENT
    Llama 3.2 Version Release Date: September 25, 2024
    ...


C:\Users\Tesla Laptop>
```

Figure 12: ollama show llama3.2:1b.

```
C:\Users\Tesla Laptop>ollama show qwen2.5:1.5b
  Model
    architecture        qwen2
    parameters          1.5B
    context length      32768
    embedding length    1536
    quantization        Q4_K_M
    template source     go_template

  Capabilities
    completion
    tools

  System
    You are Qwen, created by Alibaba Cloud. You are a helpful assistant.

  License
    Apache License
    Version 2.0, January 2004
    ...


C:\Users\Tesla Laptop>
```

Figure 13: ollama show qwen2.5:1.5b.

Note that the two models use different quantization levels. Q8_0 stores weights with 8 bits and Q4_K_M uses roughly 4 bits, so llama3.2:1b has fewer parameters but a larger file than qwen2.5:1.5b. This matters when comparing memory use and speed.

4.4 Command line check

Before building the interface, each model was tested directly in the terminal with ollama run, using the prompt "explain voltage in two sentences" and leaving with /bye. Both models answered correctly, which confirmed that the Ollama runtime and the downloaded model files worked.

```
C:\Users\Tesla Laptop>ollama run llama3.2:1b
>>> explain voltage in two sentences
Voltage is the measure of the electric potential difference or "pressure" between two points in an electrical circuit, with higher voltage indicating a greater
potential difference. In simpler terms, voltage is the "force" that drives electric current through a circuit, and it's what allows electricity to flow from one
device to another.

>>> /bye

C:\Users\Tesla Laptop>
```

Figure 14: ollama run llama3.2:1b: explain voltage in two sentences.

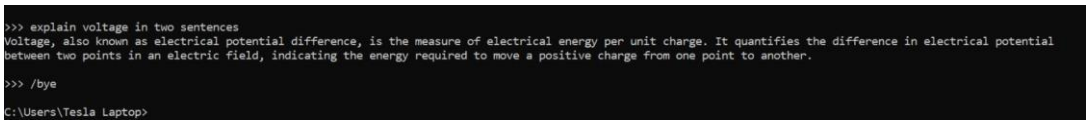


Figure 15: Second terminal test with the same prompt (qwen2.5:1.5b). The ollama run command line is not visible in this capture, so the model name is not displayed on screen.

5. Architecture and Data Flow

The system is a local pipeline. Nothing in the path to the model leaves the laptop.

#	Component	Role
1	User and browser	Types the prompt at http://localhost:8501 and reads the answer.
2	Streamlit interface (Python)	Draws the Local AI Lab page: model selector, active model banner, clear button, chat area, last response timing and log viewer.
3	Application logic (Python)	Holds the conversation in session memory, builds the request, measures time and handles errors.
4	requests library	Sends an HTTP call from the app to the local Ollama service.
5	Ollama service	Background service on the same laptop that receives the call, loads the chosen model and generates text.
6	Model files	llama3.2:1b and qwen2.5:1.5b stored on the SSD by Ollama.
7	Conversation log	Saved record of prompts, answers, model name and timing, shown in the Recent saved conversation log panel.

Data flow: the user submits a prompt, Streamlit passes it to the Python code, the code sends it through requests to Ollama, Ollama loads the selected model from disk into RAM (this is the model load time), generates the answer on the CPU or GPU, and returns it. The app displays the answer with the model name and response time, and writes the exchange to the saved log.

Processes and services involved: the Streamlit server process (python), the Ollama background service and model runner, and the browser. Files involved: requirements.txt, the Python app file, the model files, and the conversation log file.

5.1 System architecture diagram

Diagram 1 shows the workflow of the application. The browser talks only to the Streamlit app on the same laptop, the app calls the local Ollama service through the requests library, Ollama loads the selected model from the model files stored on the SSD, and each exchange is written to a local log file. No component is hosted online.

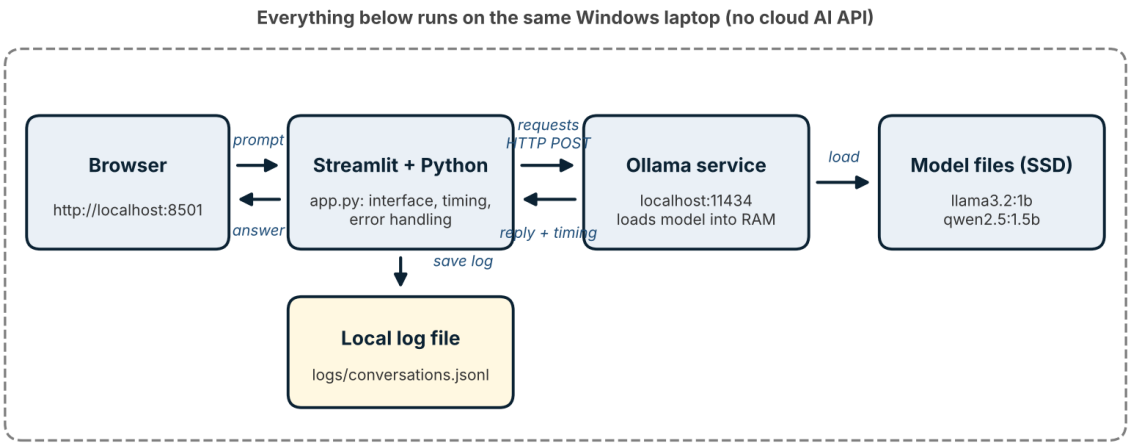


Diagram 1: System architecture and data flow of the Local AI Lab (browser, Streamlit/Python, requests, local Ollama, model files, local log).

6. Implementation

6.1 Folder structure

The project lives in C:\Users\Tesla Laptop\Documents\LocalLLMProject and is run from PowerShell inside that folder.

File / folder	Purpose
`requirements.txt`	Python dependencies: streamlit and requests
`app.py`	Streamlit interface and Ollama call (the only Python source file; it imports only the standard library, requests and streamlit)
`logs/`	Folder created automatically by app.py when the app starts
`logs/conversations.jsonl`	Saved conversation log (JSONL): one JSON line per prompt and response, written by save_log() and shown in the Recent saved conversation log panel

6.2 Streamlit interface

The interface, titled Local AI Lab with the subtitle "Two local text models, one interface, no cloud AI API", runs at localhost:8501. Its sidebar is titled Controls and contains the model dropdown, an Active model banner, a Clear current conversation button and a Last response panel. The main area contains the chat, a Recent saved conversation log expander and the prompt box "Type a question for the selected local model".

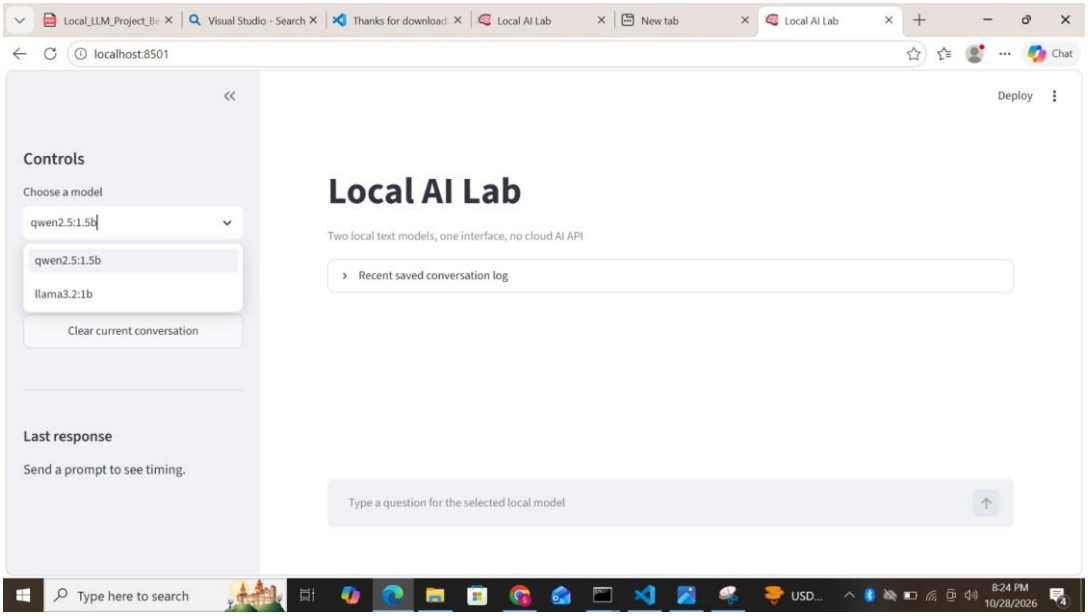


Figure 16: Local AI Lab: the model dropdown offers qwen2.5:1.5b and llama3.2:1b.

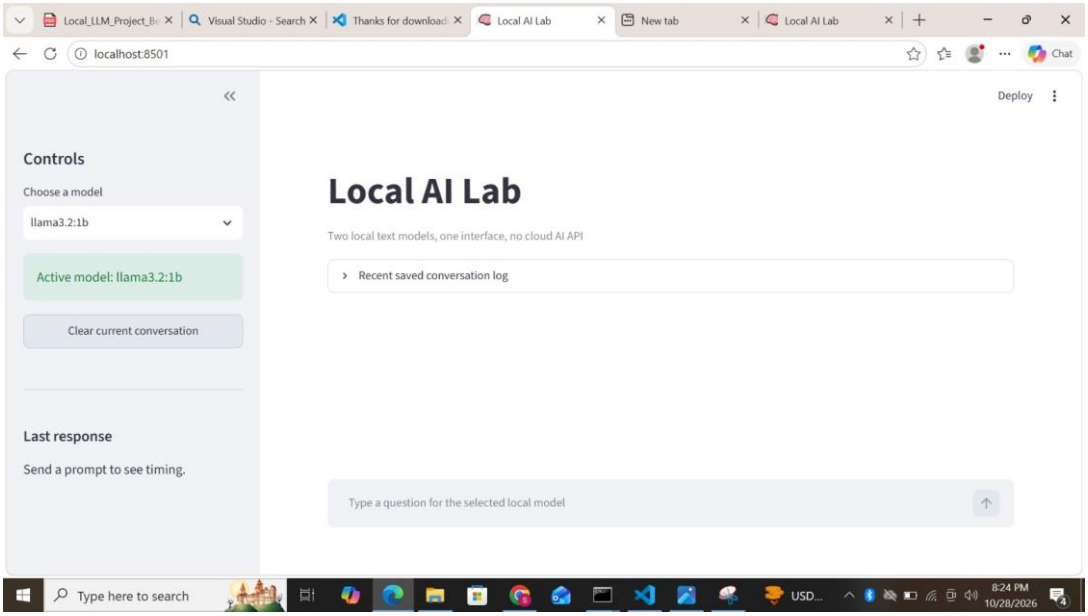


Figure 17: Local AI Lab with llama3.2:1b selected and an empty conversation.

6.3 API call, model switcher, history clearing, logging, timing and errors

- API call: the app sends the prompt to the local Ollama service over HTTP using the requests library.
- Model switcher: the dropdown changes the model name sent in the request, and the Active model banner confirms the current choice.
- History clearing: the Clear current conversation button empties the chat shown on the page.
- JSONL logging: each exchange is saved as one JSON line and the latest entries are shown in Recent saved conversation log.
- Timing: the answer is shown with a caption such as "Model: qwen2.5:1.5b | Response time: 16.92 seconds", and the sidebar shows response time and model load time.

6.4 Storage, database and credentials

Storage: the application does not use a database. The only data it saves is a local JSON Lines file, logs/conversations.jsonl, with one JSON object per prompt and response (timestamp, model, prompt, response, wall_seconds, load_seconds, output_tokens and tokens_per_second). The model files are kept in Ollama's local model storage on the SSD. No online database or external service is used.

Credentials: there is no login system. No username, password or API key is required to run or use the application; it is opened directly at http://localhost:8501.

6.5 Running instructions (fresh Windows setup)

Prerequisites: a 64-bit Windows 10 laptop (tested on 8 GB RAM), about 2.3 GB of free disk space for the two models (1.3 GB and 986 MB), and an internet connection for the one-time downloads in steps 1, 2 and 5. After the downloads, the app can run without internet (Section 9).

Step	Action	Command / detail
1	Install Python 3.12 (64-bit)	python.org installer (3.12.10 was used). Check with `python --version` and `pip --version` (Figure 4).
2	Install Ollama for Windows	Run OllamaSetup.exe from ollama.com/download/windows. Check with `ollama --version` (Figure 3).
3	Get the project files	Clone or download the GitHub repository (Section 6.6), or unzip the submitted ZIP. Open PowerShell inside the folder that contains app.py.
4	Install the Python packages	`python -m pip install -r requirements.txt`
5	Download the two models	`ollama pull llama3.2:1b` and `ollama pull qwen2.5:1.5b`. Check with `ollama list` (Figure 11).
6	Make sure Ollama is running	Ollama runs as a background service after installation. If the app reports that it cannot reach Ollama, start Ollama from the Windows Start menu.
7	Start the app	`streamlit run app.py` (if the command is not recognised, use `python -m streamlit run app.py`)
8	Open the interface	Open `http://localhost:8501` in a browser, choose a model and a prompt template, and type a prompt.

6.6 Source code and GitHub repository

The source files (app.py and requirements.txt) are in the group's GitHub repository: https://github.com/saman294/EF101-Local-LLM-Project

6.7 Member contributions

Member	Contribution
SAMAN	app coding and GitHub
DUA TOSEEF	setup, models and system measurements
SHAFIA EEMAN	testing, report writing and final package

7. Test Method

Following the project guide, the test plan was to run the same fixed set of 10 prompts on each model under identical conditions (same laptop, same power mode, same applications closed, Ollama running, one prompt at a time). Each prompt was to be timed in three runs, with the Windows Task Manager observed for RAM, CPU and GPU use while the model answers. The list below is the planned prompt set; Section 8 reports what was actually measured.

Preliminary prompts already tried in the interface were "explain ram in three sentences" (qwen2.5:1.5b) and "explain cpu in three sentences" (llama3.2:1b).

Fixed prompt set:

#	Prompt
1	From Qwen explain RAM in 3 sentences
2	From Llama explain CPU in three sentences
3	From Llama what is artificial intelligence
4	From Llama explain RAM in simple words
5	From Llama what is the difference between hardware and software
6	From Qwen explain what a local AI chatbot does in short
7	From Qwen what are the two main differences between hardware and software
8	From Qwen explain RAM in two sentences
9	From Llama what is 2+2
10	From Llama explain cpu in three sentences

In this list "From Qwen" and "From Llama" show which model the prompt was planned for. Spelling was corrected in the report ("lama" to "Llama" and "RAM and simple words" to "RAM in simple words").

8. Results

8.1 Measurements already captured

The two screenshots below are the measurements recorded so far in the interface. They used different prompts, so they are not a like-for-like speed comparison.

Model	Prompt	Response time (shown by app)	Model load time (sidebar)
qwen2.5:1.5b	explain ram in three sentences	16.92 s	41.5s
llama3.2:1b	explain cpu in three sentences	35.73 s	6.37 s

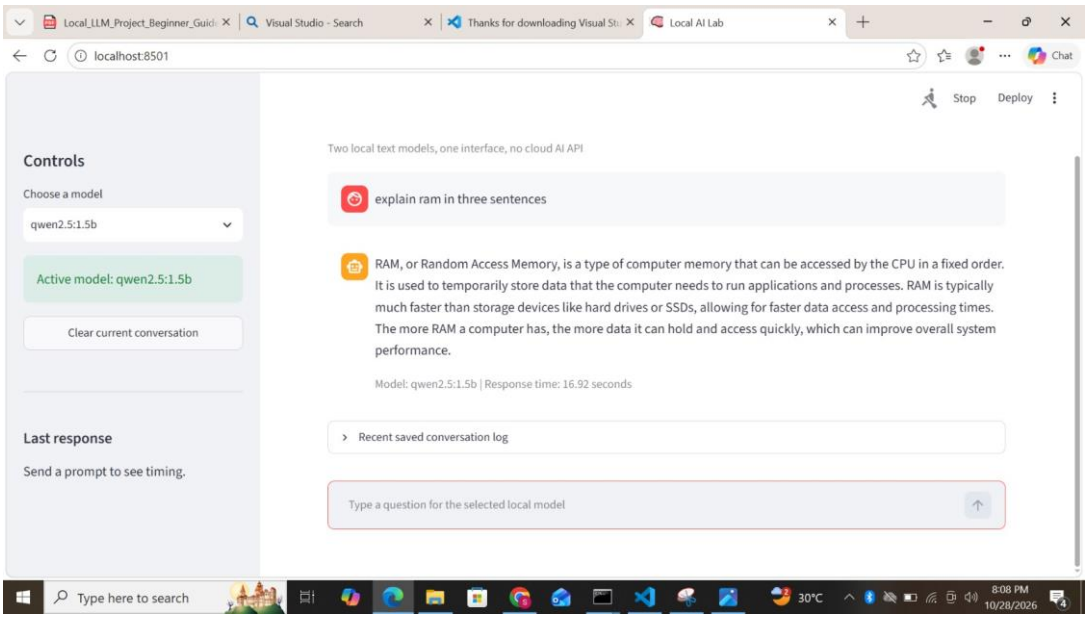


Figure 18: qwen2.5:1.5b answering "explain ram in three sentences" (response time 16.92 s).

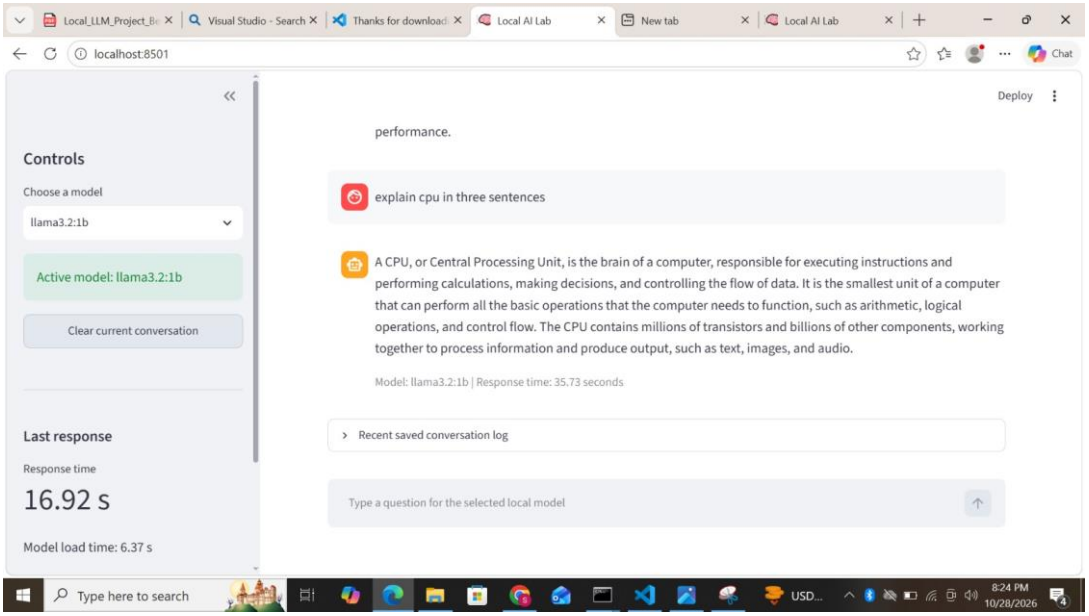


Figure 19: llama3.2:1b answering "explain cpu in three sentences" (response time 35.73 s, model load time 6.37 s).

Observation: in Figure 19 the sidebar still shows 16.92 s, which is the earlier qwen2.5:1.5b value, while the answer caption shows 35.73 s for the llama3.2:1b response.

8.2 Response quality, timing and load time (Prompt 1, 3 runs)

Readings for Prompt 1 ("explain RAM in 3 sentences") were recorded over three runs for each model, together with the average response time and the average model load time.

llama3.2:1b: Run 1 took 19.87 s, Run 2 took 5.89 s and Run 3 took 7.03 s. The average response time was 10.93 s and the average load time was 3.06 s.

qwen2.5:1.5b: Run 1 took 19.16 s, Run 2 took 4.38 s and Run 3 took 4.36 s. The average response time was 9.30 s and the average load time was 4.41 s.

Quality note: Both models provided correct, relevant and concise explanations of RAM. Run 1 was the slowest for both models.

8.3 RAM, CPU and GPU (Task Manager)

Model	Idle RAM in use (GB)	Peak RAM in use (GB)	Peak CPU (%)	GPU use (%)
llama3.2:1b	5.9	7.9	53%	1%
qwen2.5:1.5b	5.8	7.9	90-95%	7%

8.4 Selected answer examples

qwen2.5:1.5b, "explain ram in three sentences": the answer defined RAM as temporary memory used by the CPU for running applications, noted that it is much faster than hard drives or SSDs, and said that more RAM lets a computer hold and access more data quickly. Strong points: accurate and on topic. Weak point: the first sentence says "accessed in a fixed order", which describes sequential access rather than random access, and the answer is also longer than three sentences.

llama3.2:1b, "explain cpu in three sentences": the answer described the CPU as the brain of the computer that executes instructions and performs calculations, and that it handles arithmetic, logic and control flow. Weak points: calling the CPU "the smallest unit of a computer" and saying it contains "billions of other components" are inaccurate or unclear, which shows the limits of a 1B parameter model.

9. Offline Demonstration

Steps performed: (1) Airplane mode was switched on, so the laptop had no network connection; the airplane icon is visible in the taskbar in Figure 20. (2) Ollama was running in the background. (3) The Local AI Lab page was opened at localhost:8501 and llama3.2:1b was selected. (4) The prompt "what is rom in one sentence" was sent and the model answered locally. The answer caption reads "Model: llama3.2:1b | Response time: 20.44 seconds".

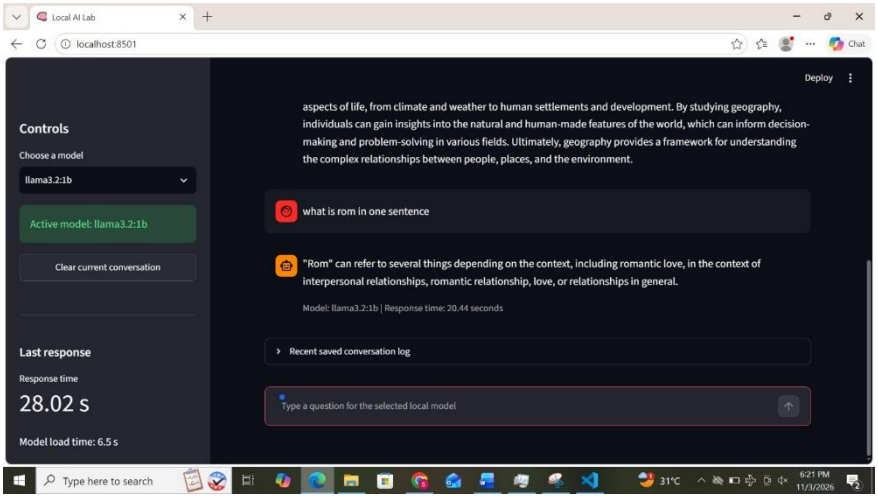


Figure 20: Local AI Lab answering a prompt with llama3.2:1b while Airplane mode is on (response time 20.44 seconds).

Why inference worked without the Internet: after ollama pull, the model weights are stored on the laptop SSD. Ollama loads them into local memory and Streamlit talks to Ollama through localhost, so the whole request path stays on the machine and no external server is needed.

What would stop working offline: downloading Python or Ollama, ollama pull for new models, pip install of new packages, checking for software updates, the Windows and browser online features, and any feature that calls an online service.

Terminology: "no cloud AI API" means the app never sends prompts to an online AI service. Fully offline operation is claimed only for the test described above, which was done with Airplane mode on.

10. Discussion

The two models are similar in size but differ in design. llama3.2:1b has 1.2B parameters, 8-bit quantization (Q8_0), a 1.3 GB file and a very large 131072-token context window. qwen2.5:1.5b has 1.5B parameters, 4-bit style quantization (Q4_K_M), a 986 MB file and a 32768-token context window.

In the captured runs, qwen2.5:1.5b answered in 16.92 s and llama3.2:1b in 35.73 s. These used different prompts and a single run each, so they should not be read as proof that one model is faster. Possible factors include the heavier Q8_0 weights of llama3.2:1b, the time to load a model into RAM (6.37 s in the sidebar for llama3.2:1b), the length of each answer and background load on a laptop with an SSD of 119 GB. For Prompt 1 (explain RAM), measured over three runs, qwen2.5:1.5b averaged 9.30 s against 10.93 s for llama3.2:1b, although its average model load time was longer (4.41 s against 3.06 s).

10.1 Problems encountered and solutions

The items below are real issues visible in the project's code and measurements. The last row is left for the group to add other problems it actually met.

Problem	Solution / status
Ollama not running, model not downloaded, or slow response when the app calls Ollama	Handled in app.py: connection errors, timeouts and HTTP 404 (model not found) each show a clear message, for example "Cannot reach Ollama. Start Ollama from the Windows Start menu" or "Model not found. Run: ollama pull <model>".
Empty prompt submitted	app.py shows the warning "Please enter a prompt before sending." and does not call the model.
Run 1 of each model was much slower (19.87 s and 19.16 s) than Runs 2 and 3	Consistent with the model being loaded from the SSD into RAM on the first request. No fix was needed; the load time is shown in the sidebar.
Sidebar timing shows the previous response (Figure 19)	The sidebar is rendered before the new answer is generated, so it may show the previous response until the page refreshes. The caption beneath the answer displays the current response time. This is a display-timing limitation.

11. Bonus / Extension Feature: Prompt Template Functionality

11.1 Introduction

An additional Prompt Template Feature was implemented in the Local AI Lab application to improve user interaction and make prompting easier. It allows users to select predefined instructions instead of writing detailed instructions manually.

11.2 Implementation Steps

1. **Created prompt templates:** Five options were added: Normal, Explain simply, Summarize, Define a term, and Step-by-step. They are stored in the PROMPT_TEMPLATES dictionary in the app file (Section 13, app.py listing).
2. **Added a dropdown menu:** A new selection menu, "Choose a prompt template", was added to the Streamlit sidebar, allowing users to choose their preferred prompt template (Figure 21).
3. **Applied template instructions:** When a user submits a question, the selected template instruction is inserted at the start of the messages sent to the local AI model as a system message. The Normal option adds no instruction.
4. **Maintained existing functionality:** The feature works with both Llama 3.2 (1B) and Qwen 2.5 (1.5B) without changing the existing model selection, response timing, or conversation logging features.
5. **Tested the feature:** Different templates were tested to verify that the models could respond according to the selected instructions (Figures 22 and 23).

11.3 Results

The Prompt Template Feature successfully added different response styles to the existing chatbot. Users can request simple explanations, summaries, definitions, or step-by-step responses through a dropdown menu.

11.4 Evidence

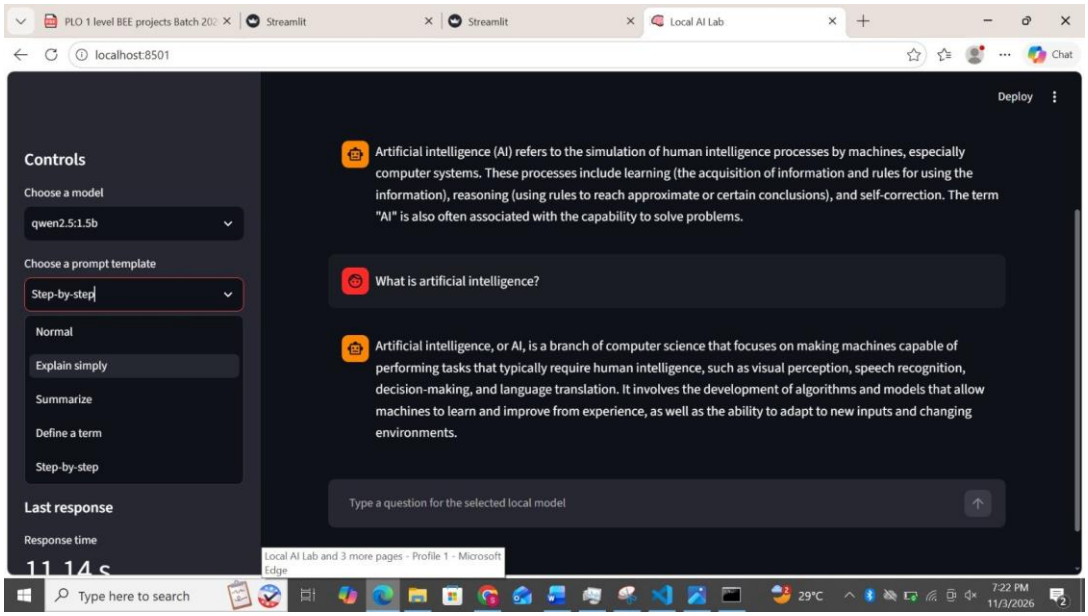


Figure 21: Local AI Lab sidebar showing the Prompt Template dropdown with the five options: Normal, Explain simply, Summarize, Define a term and Step-by-step.

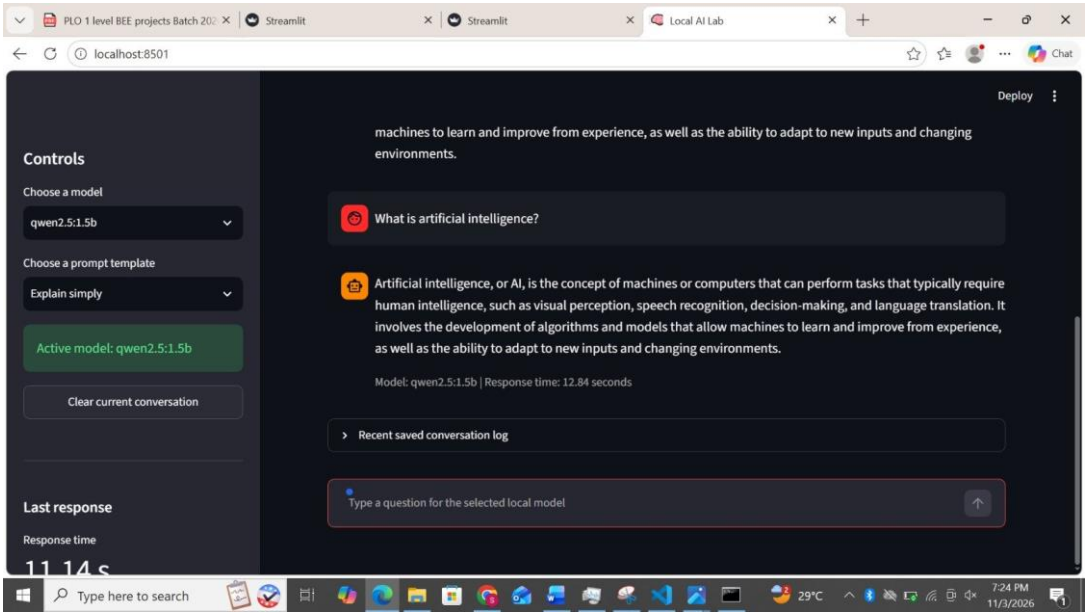


Figure 22: Response generated by qwen2.5:1.5b to "What is artificial intelligence?" using the Explain simply template (response time 12.84 seconds).

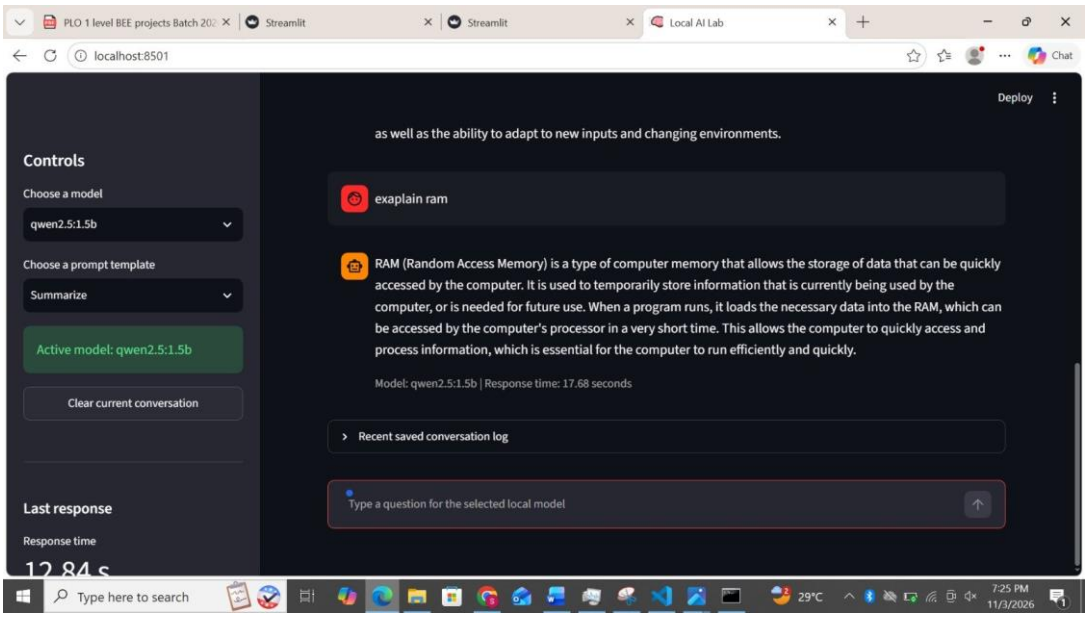


Figure 23: Response generated by qwen2.5:1.5b to "explain ram" using the Summarize template (response time 17.68 seconds).

11.5 Conclusion

This extension improved the usability of the Local AI Lab by providing predefined prompting options. It made the chatbot easier to use while retaining local model execution without relying on cloud-based AI APIs.

12. Conclusion

The group installed Python 3.12.10 and Ollama 0.40.11 on a Windows 10 Pro laptop, pulled two local models, and built a Streamlit interface that switches between them, times each response and keeps

a saved conversation log, all without a cloud AI API. The project shows that local inference is a complete computing system: the operating system, storage, memory, processor, runtime, application code and interface all affect what the user experience.

We learned how to make an offline chatbot by using multiple software and running their commands, introduction to github and report making.

13. References and Appendix

References

1. Python 3.12.10 installer: https://www.python.org/downloads/release/python-31210/
2. Python downloads: https://www.python.org/downloads/release/python-31210/
3. Ollama installer (OllamaSetup.exe) from the official Ollama GitHub release assets; project page: https://github.com/ollama/ollama
4. Ollama download and documentation: https://ollama.com/download/windows
5. Ollama model library: https://ollama.com/library
6. Streamlit documentation: https://docs.streamlit.io/
7. Requests documentation: https://requests.readthedocs.io
8. Project guide: https://dev.to/lingdas1/local-llm-guide-the-complete-series-find-your-starting-point-4do5
9. Project source code (GitHub repository):https://github.com/saman294/EF101-Local-LLM-Project

app.py (Local AI Lab, including the prompt template feature):

```
import json
import time
from datetime import datetime
from pathlib import Path

import requests
import streamlit as st

OLLAMA_CHAT_URL = "http://localhost:11434/api/chat"
MODELS = ["qwen2.5:1.5b", "llama3.2:1b"]
PROMPT_TEMPLATES = {
    "Normal": "",
    "Explain simply": "Explain the answer in simple language that a beginner can understand.",
    "Summarize": "Give a short and clear summary of the topic.",
    "Define a term": "Give a clear definition and one simple example.",
    "Step-by-step": "Explain the answer in clear, numbered steps."
}
LOG_FILE = Path("logs/conversations.jsonl")
LOG_FILE.parent.mkdir(exist_ok=True)

st.set_page_config(page_title="Local AI Lab", page_icon="🤖", layout="wide")

if "messages" not in st.session_state:
    st.session_state.messages = []
if "last_metrics" not in st.session_state:
    st.session_state.last_metrics = None



def save_log(record):
    """Append one prompt response pair to a local JSONL file."""
```

```
    with LOG_FILE.open("a", encoding="utf-8") as file:
        file.write(json.dumps(record, ensure_ascii=False) + "\n")



def read_recent_logs(limit=10):
    """Return the most recent saved prompt response pairs."""
    if not LOG_FILE.exists():
        return []
    lines = LOG_FILE.read_text(encoding="utf-8").splitlines()
    records = []
    for line in lines[-limit:]:
        try:
            records.append(json.loads(line))
        except json.JSONDecodeError:
            pass
    return records



def ask_model(model, messages):
    """Send the chat to local Ollama and return answer plus timing data."""
    start = time.perf_counter()
    response = requests.post(
        OLLAMA_CHAT_URL,
        json={"model": model, "messages": messages, "stream": False},
        timeout=300,
    )

    response.raise_for_status()
    data = response.json()
    wall_seconds = time.perf_counter() - start
    eval_count = data.get("eval_count", 0)
    eval_seconds = data.get("eval_duration", 0) / 1_000_000_000

    tokens_per_second = eval_count / eval_seconds if eval_seconds > 0 else None
    return data["message"]["content"], {
        "wall_seconds": round(wall_seconds, 2),
        "load_seconds": round(data.get("load_duration", 0) / 1_000_000_000, 2),
        "output_tokens": eval_count,
        "tokens_per_second": round(tokens_per_second, 2) if tokens_per_second else None,
    }



st.title("Local AI Lab")
st.caption("Two local text models, one interface, no cloud AI API")

with st.sidebar:
    st.header("Controls")
    selected_model = st.selectbox("Choose a model", MODELS)

    selected_template = st.selectbox(
        "Choose a prompt template",
        list(PROMPT_TEMPLATES.keys())
    )
    st.success(f"Active model: {selected_model}")
    if st.button("Clear current conversation", use_container_width=True):
        st.session_state.messages = []
        st.session_state.last_metrics = None
        st.rerun()
    st.divider()
    st.subheader("Last response")
    if st.session_state.last_metrics:
        metric = st.session_state.last_metrics
        st.metric("Response time", f"{metric['wall_seconds']} s")
        st.write(f"Model load time: {metric['load_seconds']} s")
        st.write(f"Output tokens: {metric['output_tokens']}")
        st.write(f"Tokens per second: {metric['tokens_per_second'] or 'Unavailable'}")
    else:
        st.write("Send a prompt to see timing.")

for message in st.session_state.messages:
    with st.chat_message(message["role"]):
```

```
        st.markdown(message["content"])

prompt = st.chat_input("Type a question for the selected local model")

if prompt is not None:
    if not prompt.strip():
        st.warning("Please enter a prompt before sending.")
    else:
        user_message = {"role": "user", "content": prompt.strip()}
        st.session_state.messages.append(user_message)
        with st.chat_message("user"):
            st.markdown(user_message["content"])
        try:
            request_messages = st.session_state.messages.copy()

            template_instruction = PROMPT_TEMPLATES[selected_template]

            if template_instruction:
                request_messages.insert(0, {
                    "role": "system",
                    "content": template_instruction
                })

            with st.chat_message("assistant"):
                with st.spinner(f"{selected_model} is generating locally..."):
                    answer, metrics = ask_model(selected_model, request_messages)

                st.markdown(answer)
                st.caption(f"Model: {selected_model} | Response time: {metrics['wall_seconds']}
seconds")

            st.session_state.messages.append({"role": "assistant", "content": answer})
            st.session_state.last_metrics = metrics

            save_log({
                "timestamp": datetime.now().astimezone().isoformat(timespec="seconds"),
                "model": selected_model,
                "prompt": user_message["content"],
                "response": answer,
                **metrics,
            })
        except requests.exceptions.ConnectionError:
            st.error("Cannot reach Ollama. Start Ollama from the Windows Start menu, then try again.")
        except requests.exceptions.Timeout:
            st.error("The model took too long. Wait a moment, try a shorter prompt, or restart
Ollama.")
        except requests.exceptions.HTTPError as error:
            if error.response is not None and error.response.status_code == 404:
                st.error(f"Model not found. Run: ollama pull {selected_model}")
            else:
                st.error(f"Ollama returned an error: {error}")
        except Exception as error:
            st.error(f"Unexpected error: {error}")

with st.expander("Recent saved conversation log"):
    recent = read_recent_logs(10)
    if recent:
        st.dataframe(recent, use_container_width=True, hide_index=True)
    else:
        st.info("No saved prompt response pairs yet.")
```

10.2 Evaluation Coverage and Remaining Limitations

This section separates the work documented in this report from the remaining items in the EF-101 project guide. It is included to make the evaluation scope clear and avoid presenting planned tests as completed measurements.

Evidence documented in this report

Requirement	Evidence currently documented	Assessment
Fixed prompt evaluation	A fixed set of 10 prompts is listed. Three timing runs are reported for Prompt 1 on each model; results for Prompts 2–10 are not included.	**Partial**
Repeated timing	Prompt 1 averages: Llama 3.2 1B = 10.93 s (average load 3.06 s); Qwen 2.5 1.5B = 9.30 s (average load 4.41 s).	**Partial**
RAM, CPU and GPU observations	Task Manager readings are reported for both models in Section 8.3.	**Documented**
Offline demonstration	One offline response is documented: Llama 3.2 1B answered a ROM prompt while Airplane mode was on (20.44 s). Two additional offline prompts are not evidenced in this report.	**Partial**
Conversation log	A repository check on 14 November 2026 found 16 valid JSONL records: 9 for Qwen 2.5 1.5B and 7 for Llama 3.2 1B.	**Count met at review**
Functional and failure cases	The application code includes blank-prompt handling, model selection, conversation clearing, and messages for connection, timeout, missing-model and other errors. A complete test-evidence matrix for restart and all failure cases is not included.	**Implementation shown; test evidence partial**

Items still required for full guide coverage

The full 10-prompt comparison on both models, three representative timing/resource runs beyond the single prompt set documented here, and a three-prompt offline demonstration are not fully evidenced in the current report. A complete record of restart and failure-case testing is also not provided. These items should be completed and supported with genuine measurements or screenshots before describing every guide requirement as fully met.

Interpretation: The measured results in Section 8 are valid as the observations recorded for the stated prompts and runs. They should not be generalized into a claim that one model is universally faster or better. The comparison is limited to the two model versions and the hardware used for this project.

Report Correction Addendum

This addendum supplies corrected text for the conclusion and references in the original report. It does not claim that missing experiments were performed.

Replacement conclusion

Through this project, we learned how to develop and run a locally hosted AI chatbot, use software tools and command-line instructions, work with GitHub, and prepare technical documentation. We also learned that response speed and resource use depend on model size, quantization, model loading and the computer's available hardware. The evaluation remains limited to the measurements and offline test documented in this report; the remaining tests are identified in Section 10.2.

Cleaned reference list

1. Python 3.12.10 release: https://www.python.org/downloads/release/python-31210/

2. Ollama project and source: https://github.com/ollama/ollama

3. Ollama for Windows: https://ollama.com/download/windows

4. Ollama model library: https://ollama.com/library

5. Streamlit documentation: https://docs.streamlit.io/

6. Requests documentation: https://requests.readthedocs.io/

7. Local LLM project guide: https://dev.to/lingdas1/local-llm-guide-the-complete-series-find-your-starting-point-4do5

8. Project source code (GitHub repository): https://github.com/saman294/EF101-Local-LLM-Project

Appendix: Complete app.py Source Listing

This listing was checked against the app.py file retrieved from the project's GitHub repository. The page icon is written as the supported Streamlit icon name 'brain' to keep the listing readable in standard PDF fonts.

```
import json
import time
from datetime import datetime
from pathlib import Path

import requests
import streamlit as st

OLLAMA_CHAT_URL = "http://localhost:11434/api/chat"
MODELS = ["qwen2.5:1.5b", "llama3.2:1b"]
PROMPT_TEMPLATES = {
    "Normal": "",
    "Explain simply": "Explain the answer in simple language that a beginner can understand.",
    "Summarize": "Give a short and clear summary of the topic.",
    "Define a term": "Give a clear definition and one simple example.",
    "Step-by-step": "Explain the answer in clear, numbered steps."
}
LOG_FILE = Path("logs/conversations.jsonl")
LOG_FILE.parent.mkdir(exist_ok=True)

st.set_page_config(page_title="Local AI Lab", page_icon="brain", layout="wide")

if "messages" not in st.session_state:
    st.session_state.messages = []
if "last_metrics" not in st.session_state:
    st.session_state.last_metrics = None


def save_log(record):
    """Append one prompt response pair to a local JSONL file."""
    with LOG_FILE.open("a", encoding="utf-8") as file:
        file.write(json.dumps(record, ensure_ascii=False) + "\n")


def read_recent_logs(limit=10):
    """Return the most recent saved prompt response pairs."""
    if not LOG_FILE.exists():
        return []
    lines = LOG_FILE.read_text(encoding="utf-8").splitlines()
    records = []
    for line in lines[-limit:]:
        try:
            records.append(json.loads(line))
        except json.JSONDecodeError:
            pass
    return records


def ask_model(model, messages):
    """Send the chat to local Ollama and return answer plus timing data."""
    start = time.perf_counter()
    response = requests.post(
        OLLAMA_CHAT_URL,
        json={"model": model, "messages": messages, "stream": False},
        timeout=300,
    )
    response.raise_for_status()
    data = response.json()
    wall_seconds = time.perf_counter() - start
    eval_count = data.get("eval_count", 0)
    eval_seconds = data.get("eval_duration", 0) / 1_000_000_000

    tokens_per_second = eval_count / eval_seconds if eval_seconds > 0 else None
    return data["message"]["content"], {
        "wall_seconds": round(wall_seconds, 2),
        "load_seconds": round(data.get("load_duration", 0) / 1_000_000_000, 2),
        "output_tokens": eval_count,
        "tokens_per_second": round(tokens_per_second, 2) if tokens_per_second else None,
    }


st.title("Local AI Lab")
st.caption("Two local text models, one interface, no cloud AI API")

with st.sidebar:
    st.header("Controls")
    selected_model = st.selectbox("Choose a model", MODELS)
    selected_template = st.selectbox(
        "Choose a prompt template",
```

```
        list(PROMPT_TEMPLATES.keys())
    )
    st.success(f"Active model: {selected_model}")
    if st.button("Clear current conversation", use_container_width=True):
        st.session_state.messages = []
        st.session_state.last_metrics = None
        st.rerun()
    st.divider()
    st.subheader("Last response")
    if st.session_state.last_metrics:
        metric = st.session_state.last_metrics
        st.metric("Response time", f"{metric['wall_seconds']} s")
        st.write(f"Model load time: {metric['load_seconds']} s")
        st.write(f"Output tokens: {metric['output_tokens']}")
        st.write(f"Tokens per second: {metric['tokens_per_second'] or 'Unavailable'}")
    else:
        st.write("Send a prompt to see timing.")

for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])

prompt = st.chat_input("Type a question for the selected local model")

if prompt is not None:
    if not prompt.strip():
        st.warning("Please enter a prompt before sending.")
    else:
        user_message = {"role": "user", "content": prompt.strip()}
        st.session_state.messages.append(user_message)
        with st.chat_message("user"):
            st.markdown(user_message["content"])
        try:
            request_messages = st.session_state.messages.copy()
            template_instruction = PROMPT_TEMPLATES[selected_template]
            if template_instruction:
                request_messages.insert(0, {
                    "role": "system",
                    "content": template_instruction
                })
            with st.chat_message("assistant"):
                with st.spinner(f"{selected_model} is generating locally..."):
                    answer, metrics = ask_model(selected_model, request_messages)
                st.markdown(answer)
                st.caption(f"Model: {selected_model} | Response time: {metrics['wall_seconds']} seconds")
            st.session_state.messages.append({"role": "assistant", "content": answer})
            st.session_state.last_metrics = metrics
            save_log({
                "timestamp": datetime.now().astimezone().isoformat(timespec="seconds"),
                "model": selected_model,
                "prompt": user_message["content"],
                "response": answer,
                **metrics,
            })
        except requests.exceptions.ConnectionError:
            st.error("Cannot reach Ollama. Start Ollama from the Windows Start menu, then try again.")
        except requests.exceptions.Timeout:
            st.error("The model took too long. Wait a moment, try a shorter prompt, or restart Ollama.")
        except requests.exceptions.HTTPError as error:
            if error.response is not None and error.response.status_code == 404:
                st.error(f"Model not found. Run: ollama pull {selected_model}")
            else:
                st.error(f"Ollama returned an error: {error}")
        except Exception as error:
            st.error(f"Unexpected error: {error}")

with st.expander("Recent saved conversation log"):
    recent = read_recent_logs(10)
    if recent:
        st.dataframe(recent, use_container_width=True, hide_index=True)
    else:
        st.info("No saved prompt response pairs yet.")
```